

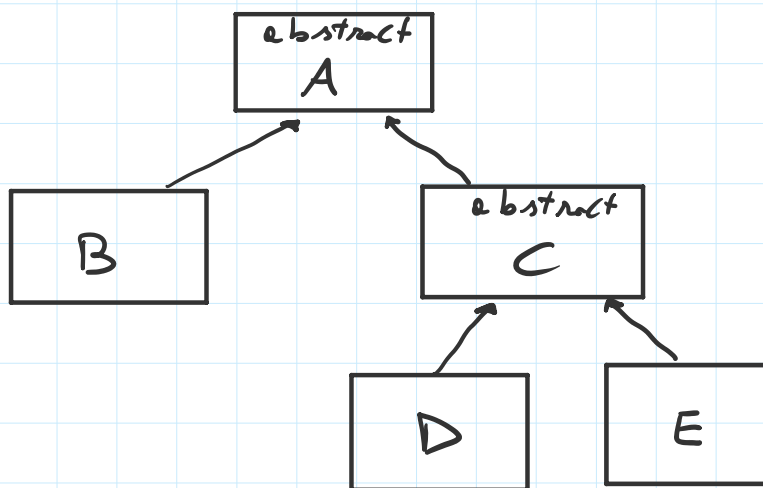
Classi Astratte

Una classe astratta non può essere istanziata

Può essere usata come classe base nella definizione di nuove classi

Utile quindi come "modello" per la definizione di nuove classi.

Posso creare riferimenti di tipo classe astratta



Posso creare istanze di B, D, E

Non posso creare istanze di A, C

Posso creare riferimenti di tipo A, B, C, D, E

Esempio:

```

abstract class OggettoGrafico {
    int x, y;
    void sposta(int nuova x, int nuova y) {
        -
    }
  }
  
```

```

    }
}

```

```

class Bottone extends OggettoGrafico {
    :
}
class CasellaDiTesto extends OggettoGrafico {
    :
}

```

Non possono esistere oggetti grafici che non siano bottone o casella di testo.

```

OggettoGrafico og1 = new OggettoGrafico(); //ERRORE

```

```

OggettoGrafico og2 = new Bottone(); // OK

```

```

og2.sposta(100, 200); // OK, ereditato
                        da OggettoGrafico

```

In una classe estratta alcuni metodi possono essere inolchi come abstract

Per esempio:

```

abstract class OggettoGrafico {
    int x, y;
    void sposta(int nuova x, int nuova y) {

```

```

int x, y;
void sposta (int nuova x, int nuova y) {
    ==
}
abstract void disegna ();
}

```

↑ Nota: c'è ; non {}

```

class Bottone extends OggettoGrafico {
    void disegna () {
        ==
    }
}

```

← definizione di disegna

```

class CasellaDiTesto extends OggettoGrafico {
    String testo;
    void disegna () {
        ==
    }
    void altroMetodo () {
        ==
    }
}

```

Se c'è almeno un metodo abstract la classe deve essere abstract.

```

OggettoGrafico og1 = new Bottone ();
OggettoGrafico og2 = new CasellaDiTesto ();
og1.sposta (100, 200);

```

Oggetto Grafico og2: new Casella Di Testo(1);
og1. sposta(100, 200);
og1. disegna(); // viene eseguito il disegno
d' Bottone

```
abstract class X {
```

```
void m1() {
```

$$m_2(1);$$

Posso chiamare
metodi astratti

3

```
abstract void m2();
```

1

3

Classi e metab. I grado

Una classe finale non può essere estesa

final class Y {

三

3

non posso scrivere

```
class X extends Y {
```

...

3

is more

E' possibile definire metodi final

```
class C {  
    final void mioMetodo () {  
        ==  
    }  
    :  
}  
class D extends C {  
    :  
    void mioMetodo () {  
        ==  
    }  
}
```

OK

errore: metodo final, non può essere ridefinito

Quando una classe è final allora tutti i suoi metodi sono final.

Nel mondo Java le classi e i metodi final sono pochi

```
final boolean controllaPassword (String p) {  
    ==  
}
```

impedisce che venga ridefinito in modo da restituire sempre true

Interfacce

In Java un'interfaccia è una raccolta di metodi abstract

Un'interfaccia definisce un protocollo di comportamento a cui le classi possono decidere di aderire

Possiamo definire nuove interfacce

Possiamo scrivere classi che implementano interfacce

Per definire un'interfaccia si usa la parola chiave interface

Esempio:

```
public interface Comparabile {
```

```
    int compare (Object o);
```

```
}
```

↑ implicitamente public e abstract

↑
in Comparabile.java

```
$ javac Comparabile.java
```

produce Comparabile.class

Una classe che implementa un'interfaccia deve fornire l'implementazione di tutti i metodi dell'interfaccia.

Esempio:

```
public class Data implements Comparabile {
    int giorno;
    int mese;
    int anno;

    public Data (int g, int m, int a) {
        giorno = g;
        mese = m;
        anno = a;
    }

    public int compara (Object o) {
        Data d = (Data) o;
        if (anno != d.anno)
            return anno - d.anno;
        if (mese != d.mese)
            return mese - d.mese;
        return giorno - d.giorno;
    }
}
```

Posso creare riferimenti al tipo interfaccia
Non posso creare oggetti al tipo interfaccia

Riferimenti al tipo interfaccia possono puntare
a oggetti che sono istanze di classi che implementano
l'interfaccia.

```
Data d1 = new Data(10, 5, 1950);
```

```
Data d2 = new Data(11, 6, 2000);
```

```
Comparabile e1 = d1;
```

```
int r = e1.compara(d2);
```

↑ posso chiamare i soli
metodi dell'interfaccia

Anche la classe ContoBancario può aderire
allo stesso protocollo di comportamento

```
public class ContoBancario implements Comparabile {
```

```
    ==
```

```
    public int compara(Object o) {
```

```
        ContoBancario c = (ContoBancario) o;
```

```
        if (bilancio < c.bilancio)
```

```
            return -1;
```

```
        if (bilancio > c.bilancio)
```

```
            return 1;
```

```
        return 0;
```

```
    }  
}
```

←

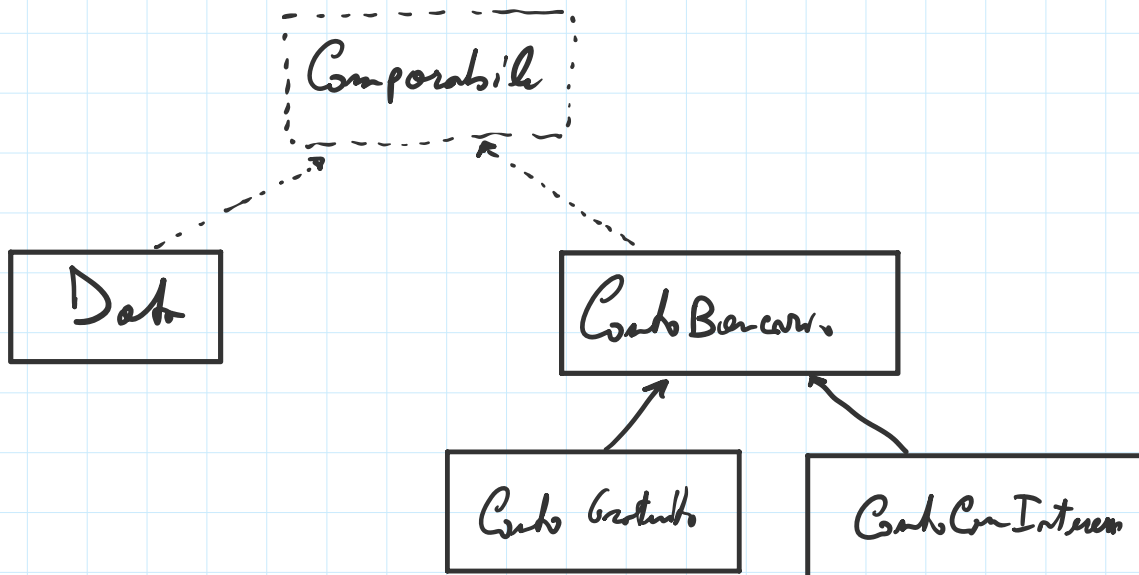
```
Comparabile x = new ContoBancario(...);
```

```
ContoBancario y = new ContoBancario(...);
```

```
int w = x.compara(y);
```

x.compara (new Data (...));

↑ ClassCastException: un Data non può essere convertito a ContoBanca



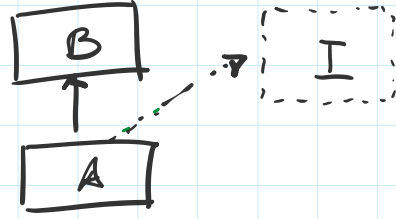
Una classe può implementare tutte le interfacce che vuole

```
class X implements Interf1, Interf2, Interf3 {  
    :  
}
```

Una classe può estendere un'altra e al tempo stesso implementare una o più interfacce

```
class A extends B implements I {
```

}



Se una classe implementa più interfacce
deve fornire un'implementazione di tutti i
metodi di tutte le interfacce

```
public interface Interf1 {
```

```
    void m1();
```

```
}
```

```
public interface Interf2 {
```

```
    void m2();
```

```
    int m3(int x);
```

```
}
```

```
public interface Interf3 {
```

```
    float m4(float f1, float f2);
```

```
}
```

```
class X implements Interf1, Interf2, Interf3 {
```

```
    void m1() {  
        =
```

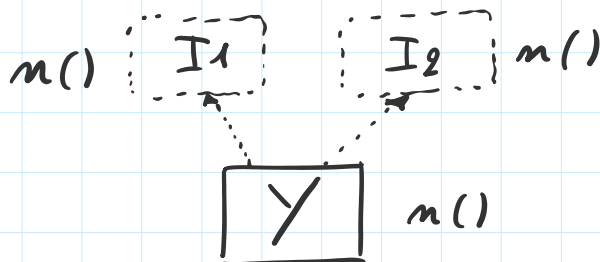
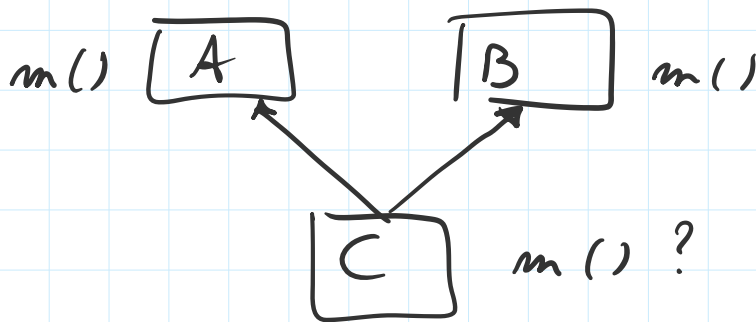
```
    }
```

```
    void m2() {  
        =
```

```

    }
    int m3(int x){
        =
    }
    float m4(float f1, float f2){
        =
    }
    {
        {
            =
        }
    }

```



```

public interface Closable {
    void close();
}

```

```

public class Flume implements Closable {
    =
    public void close() {
        =
    }
}

```

```
{  
public class FlussFile extends Fluss {
```

Però nella funzione close() non
non è obbligatorio.

```
}
```

```
public abstract class Fluss implements Closeable {  
    =
```

```
}
```

```
public class FlussFile extends Fluss {
```

```
    public void close() {  
        =
```

```
    }
```

```
    =
```

```
}
```

Un'interfaccia può contenere delle costanti

Esempio:

```
public interface Verbose {
```

```
    int SILENT = 0;
```

```
    int NORMAL = 1;
```

```
    int VERBOSE = 2;
```

```
    void setVerbosity(int level);
```

```
    int getVerbosity();
```

```
}
```

} sono implicitamente
public, static e final

una classe che implementa un'interfaccia
"eredita" le costanti dell'interfaccia e le può usare.

```
public class Logger implements Verbose {  
    private int level;  
    :  
    public void setVerbosity(int l) {  
        level = l;  
    }  
    public int getVerbosity() {  
        return level;  
    }  
    public void log(String m) {  
        if (level == SILENT)  
            return;  
        if (level == NORMAL)  
            ;  
    }  
}
```

posso usare
il nome semplice
della costante

In una classe che non implementa l'interfaccia
devo usare la forma

NomeDellInterfaccia . nomeDellaCostante

per esempio

Verbose.SILENT

Il linguaggio prevede l'interfaccia
java.lang.Comparable

int compareTo(-)

java.util.Arrays

public static void sort(Object[] v)

Tipi enumerazione

Assomigliano ai tipi enumerazione del C/C++

Tutti i valori sono noti nel momento in cui il tipo viene dichiarato

Il numero di valori possibili è non grandissimo

Nella forma più semplice sono molto simili a C/C++

```
class Main {
```

```
    enum Colore {VERDE, GIALLO, ROSSO};
```

```
    public static void main(String[] args) {
```

```
        Colore c1 = Colore.VERDE;
```

```
        // Colore c2 = GIALLO; //ERRORE,
```

```
        usare Colore.GIALLO
```

```
        switch(c1) {
```

```
            case VERDE:
```

```
                System.out.println("Vai");
```

```
                break;
```

```
            case GIALLO:
```

```
                System.out.println("Rallenta");
```

```
                break;
```

```
            case ROSSO:
```

```
                System.out.println("Fermati");
```

```
        }
```

```
    }
```

```
}
```

viene definito il
tipo enum

Colore

valori possibili:

VERDE, GIALLO, ROSSO

qui niente Colore.

In effetti gli enum sono delle classi

che estendono java.lang.Enum

Ereditano la tale classe un certo numero di metodi:

`public String toString()` restituisce il nome dell'enumeratore (per esempio "GIALLO")

`public static E[] values()` restituisce un array contenente tutti i valori

`public static E valueOf(String n)` restituisce l'enumeratore corrispondente alla stringa `n`
Se non c'è corrispondenza lancia `IllegalArgumentException`

E è il tipo enumerazione

`public int ordinal()` restituisce il n° d'ordine (0 per VERDE, 1 per GIALLO, ...)

Esempio

```
import java.util.Scanner;
```

```
class Main {
```

```
    enum Colore {VERDE, GIALLO, ROSSO};
```

```
    public static void main(String[] args) {
```

```
        Colore c1 = Colore.VERDE;
```

```
        System.out.println(c1);
```

```
        for(Colore c2: Colore.values())
```

```
            System.out.println(c2 + " " + c2.ordinal());
```

```
        Scanner sc = new Scanner(System.in);
```

```
        String parola = sc.next();
```

```
        Colore c3 = Colore.valueOf(parola);
```

```
        System.out.println("Il colore è " + c3);
```

Stampa VERDE

restituisce un array con i valori

```

Scanner sc = new Scanner(System.in);
String parola = sc.next();
Colore c3 = Colore.valueOf(parola);
System.out.println("Hai inserito: " + c3);
}
}

```

testiamo
un array
con i valori
di colore

PA2021_enum2

```

1 import java.util.Scanner;
2
3 class Main {
4
5     enum Colore {VERDE, GIALLO, ROSSO};
6
7     public static void main(String[] args) {
8
9         Colore c1 = Colore.VERDE;
10        System.out.println(c1);
11
12        for(Colore c2: Colore.values())
13            System.out.println(c2 + " " + c2.ordinal());
14
15        Scanner sc = new Scanner(System.in);
16        String parola = sc.next();
17        Colore c3 = Colore.valueOf(parola);
18        System.out.println("Hai inserito: " + c3);
19    }
20 }

```

```

> javac -classpath ./run_dir/junit-4.12.jar:./run_dir/hamcrest-1.3.jar:./run_dir/json-simple-1.1.1.jar -d . Main.java
> java -classpath ./run_dir/junit-4.12.jar:./run_dir/hamcrest-core-1.3.jar:./run_dir/json-simple-1.1.1.jar Main
VERDE
VERDE 0
GIALLO 1
ROSSO 2
VERDE
Hai inserito: VERDE
>

```

Essendo i tipi enum delle classi è possibile ridefinire il comportamento di metodi.

Qui ridefiniamo toString() in modo da avere un output a video che ci piace di più.

```

enum Colore2 {
    VERDE, GIALLO, ROSSO;

    public String toString() {
        switch(this) {
            case VERDE:
                return "Verde";
            case GIALLO:
                return "Giallo";
            case ROSSO:
                return "Rosso";
            default:
                return null;
        }
    }
}

```

forma particolare
di definizione di classe

valori possibili
ridefinisce toString()

in una file
P.R. 9 mare

Colore2.java

```
class Main {  
  
    public static void main(String[] args) {  
        Colore2 c = Colore2.ROSSO;  
        System.out.println("Colore " + c);  
    }  
}
```

[PA2021_enum3](#)



Essendo delle classi è possibile aggiungere stato e/o aggiungere metodi.
Esempio enum ColoreRGB che mantiene delle informazioni e ha metodi in più.

```
enum ColoreRGB {  
  
    VERDE(0, 1, 0), GIALLO(1, 1, 0), ROSSO(1, 0, 0);  
  
    private final double[] componenti;  
  
    ColoreRGB(double r, double g, double b) {  
        componenti = new double[]{r, g, b};  
    }  
  
    public double luminosita(){  
        return 0.2126*componenti[0] +  
               0.7152*componenti[1] +  
               0.0722*componenti[2];  
    }  
  
    private String comp() {  
        return "(R="+componenti[0] +  
            ", G=" + componenti[1] +  
            ", B=" + componenti[2] + ")";  
    }  
}
```

enum i
valori possibili
valori che passano
al costruttore

costruttore

```

    public String toString() {
        String s = null;
        switch(this) {
            case VERDE: s = "Verde"; break;
            case GIALLO: s = "Giallo"; break;
            case ROSSO: s = "Rosso"; break;
        }
        return s + comp();
    }
}

```

[PA2021 enum4](#)

